# Introduction to Soft Computing & Evolutionary Computation

---

# 1. Introduction to Soft Computing

### What is Soft Computing?

Soft Computing refers to a set of computational techniques that can **deal with imprecision, uncertainty, partial truth, and approximation**.
It is inspired by **human reasoning and biological systems**.

### Why Soft Computing?

- Many real-world problems do not have exact mathematical models (e.g., weather prediction, medical diagnosis).
- It provides **robust, flexible, low-cost solutions**.
- Works well even when input data is uncertain or incomplete.

### Main Components of Soft Computing

1. **Fuzzy Logic**
2. **Neural Networks**
3. **Evolutionary Computation** (GA, PSO, ACO)
4. **Probabilistic Reasoning**

---

# 2. Fuzzy Logic

Fuzzy Logic is a method that allows machines to **reason like humans**, using **degrees of truth** rather than strict True/False values.

---

## 2.1 Membership Functions (MFs)

### Definition:

A Membership Function maps an input value to a degree of belonging between **0 and 1**.

### Common Types

1. **Triangular MF**
   Simple, defined by three parameters (a, b, c).
2. **Trapezoidal MF**
   Has flat top; more flexible.
3. **Gaussian MF**
   Smooth curve, good for natural variations.
4. **Sigmoid MF**
   Useful for slow/fast growth patterns.

### Example:

Temperature = 30°C

- Membership in "Hot" = 0.7
- Membership in "Warm" = 0.3

---

## 2.2 Fuzzy Rules and Reasoning

### Fuzzy Rule

These are **IF–THEN statements** written in natural language.

**Example:**

- IF temperature is *High* THEN fan speed is *Fast*
- IF humidity is *Low* AND temperature is *Medium* THEN fan speed is *Medium*

### Reasoning Process

1. **Fuzzification** – Convert crisp inputs to fuzzy values
2. **Rule Evaluation** – Apply fuzzy rules on the fuzzified inputs
3. **Aggregation** – Combine outputs from all rules
4. **Defuzzification** – Convert final fuzzy output to crisp value

### Common Defuzzification Methods

- Centroid Method (most popular)
- Mean of Maximum
- Weighted Average

---

## 2.3 Applications of Fuzzy Logic

### Home Appliances

- Washing machines (adjust wash time)
- Air conditioners (adjust fan speed)
- Refrigerators (humidity control)

### Automobiles

- Automatic gear control
- Anti-lock braking system

### Healthcare

- Medical diagnosis systems
- Patient monitoring

### Industry

- Process control
- Quality inspection

### Other Domains

- Pattern recognition
- Robotics & navigation
- Risk assessment

---

# 3. Evolutionary Computation

Evolutionary Computation is inspired by **biological evolution and natural selection**.
These algorithms use populations, fitness functions, mutations, and survival strategies.

Major types:

- **Genetic Algorithms (GA)**
- **Swarm Intelligence**
- **Ant Colony Optimization (ACO)**

---

# 4. Swarm Intelligence

Swarm Intelligence is inspired by the **collective behavior of animals**, such as birds, fish, and insects.

### Key Features

- No central controller
- Individuals follow simple rules
- Collective behavior emerges automatically
- Robust and scalable

### Popular Swarm-Based Techniques

1. **Particle Swarm Optimization (PSO)**
   Inspired by bird flocking/fish schooling.
2. **Ant Colony Optimization (ACO)**
   Inspired by ant foraging behavior.

### Applications

- Routing
- Scheduling
- Clustering
- Feature selection
- Optimization in engineering

---

# 5. Ant Colony Optimization (ACO)

ACO is inspired by **how real ants find the shortest path** between their colony and food.

---

## 5.1 Key Concepts

### Pheromone Trails

Ants deposit chemical pheromones on paths.
Stronger pheromone → higher probability an ant will follow the path.

### Probabilistic Path Selection

Ants choose paths based on:

- Amount of pheromone
- Heuristic information (visibility, distance)

### Pheromone Update

- **Deposit**: Good (short) paths receive more pheromone.
- **Evaporation**: Old pheromone decreases to avoid early convergence.

---

## 5.2 ACO Algorithm Steps

1. **Initialization**
    - Set pheromone level on all edges.
2. **Solution Construction**
    - Each ant constructs a solution by moving through nodes based on probability.
3. **Pheromone Update**
    - Increase pheromone on good paths
    - Evaporate pheromone globally
4. **Termination**
    - Stop when maximum iterations reached or solution stabilizes.

---

## 5.3 Applications of ACO

### Routing & Network Problems

- Internet routing
- Vehicle routing problem
- Shortest path

### Optimization

- Job scheduling
- Resource allocation
- Project planning

### Engineering

- VLSI design
- Image segmentation
- Feature selection in ML

# Ant Colony Optimization (ACO) for Solving an Optimization Problem

Ant Colony Optimization (ACO) is a nature-inspired optimization algorithm based on **how real ants find the shortest path** from their colony to food by depositing **pheromones**.

To understand ACO, consider a general **optimization problem**, such as finding the **shortest path**, **best route**, or **minimum cost solution**.

---

# 1. How ACO Solves an Optimization Problem

### Step 1: Problem Representation

The optimization problem is converted into:

- **Nodes** (states or cities or components)
- **Edges** (connections with some cost/distance)

Example:
In a traveling salesman problem (TSP),

- Cities → nodes
- Path between cities → edges

Each edge is assigned:

- **Initial pheromone level $\tau$**
- **Heuristic visibility $\eta$** (e.g., 1/distance)

# Step 2: Ants Construct Solutions

Each ant starts at a random node and builds a solution by selecting the next node based on a **probabilistic rule**:

$$P_{ij} = \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{k \in allowed} \tau_{ik}^{\alpha} \eta_{ik}^{\beta}}$$

Where:

- $\tau_{ij}$: pheromone on edge (i,j)
- $\eta_{ij}$: heuristic value (e.g., 1/distance)
- $\alpha$: importance of pheromone
- $\beta$: importance of heuristic information

**Higher pheromone + shorter distances → higher probability.**

Thus ants explore multiple solution paths.

# Step 3: Evaluate Solutions

Each ant completes a full solution (e.g., full route).
Calculate its **quality**:

Lk=cost of solution built by ant k

Better solutions → lower cost.

# 2. Pheromone Update Mechanism

Pheromone update has **two parts**:

---

## (A) Pheromone Evaporation

After all ants complete their solutions, pheromones are reduced:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij}$$

Where $\rho$ is the evaporation rate ($0 < \rho < 1$).

**Purpose:**

- Prevent unlimited accumulation of pheromone
- Encourage exploration of new paths
- Avoid getting stuck in bad paths

## (B) Pheromone Deposit

Only good solutions deposit pheromones.
Ant kkk adds pheromone on every edge it used:

$$\tau_{ij} \leftarrow \tau_{ij} + \frac{Q}{L_k}$$

Where:

- $Q$: constant
- $L_k$: cost of the solution

**Better solutions (low cost) deposit more pheromone.**

Some variants use:

- Only the best ant
- Best ant of the iteration
- Best ant of all iterations

---

# 3. Solution Convergence in ACO

ACO converges toward an optimal or near-optimal solution due to repeated:

- Exploration (initial random movement)
- Exploitation (following strong pheromone trails)

### Mechanism of Convergence:

1. **Early iterations**
   Ants explore many paths → pheromone is weak and uniform.
2. **Middle iterations**
   Shorter/better paths accumulate **more pheromone**.
   Ants start to prefer these paths.

3. **Later iterations**
   Most ants repeatedly choose the best path due to:
   - High pheromone concentration
   - Strong heuristic guidance

This leads to **convergence**, where:

- One solution path becomes dominant
- Its pheromone level stabilizes

### Stopping criteria:

ACO stops when:

- Maximum iterations reached
- Best solution does not improve for many iterations
- Pheromone trails become stable

# 4. Simple Example (Shortest Path)

Suppose ants must find the shortest path from A to E.
Ants try all possible routes (A→B→E, A→C→D→E…).
Paths with **shorter distance** get more pheromone.

In later iterations:

- Most ants choose the shortest path (e.g., A→B→E)
- Pheromone on this path grows
- Algorithm converges to the optimal path

# Fuzzy Logic Architecture (with Example)

Fuzzy Logic Architecture is made of **four main stages**:

1. **Fuzzification**
2. **Rule Base**
3. **Inference Engine**
4. **Defuzzification**

This architecture helps convert **crisp (numerical)** inputs into **human-like reasoning** and provides a meaningful **crisp output**.

---

# 1. Fuzzification

**What it does:**

Converts exact numerical input values into fuzzy values (degrees of membership between **0 and 1**).

**How:**

Uses **Membership Functions (MFs)** like triangular, trapezoidal, Gaussian, etc.

# Membership Functions (MFs) Used in Fuzzification

Membership Functions measure **how much** an input belongs to a fuzzy set (value between 0 and 1).

Below are the most commonly used MFs:

---

## 1. Triangular Membership Function (TriMF)

**Shape:**

A triangle defined by 3 points (a, b, c)

**Formula:**

$$\mu(x) = \begin{cases} 0 & x < a \\ \frac{x-a}{b-a} & a \le x \le b \\ \frac{c-x}{c-b} & b \le x \le c \\ 0 & x > c \end{cases}$$

**Example:**

Temperature fuzzy set "Medium" defined by (20, 30, 40).

Input: **x = 35**

$$\mu(35) = \frac{40-35}{40-30} = \frac{5}{10} = 0.5$$

### Interpretation:

At 35°C, temperature belongs to **"Medium" with 0.5 membership**.

# 2. Trapezoidal Membership Function (TrapMF)

### Shape:

A trapezoid defined by 4 points (a, b, c, d)

### Useful for:

Flat regions where full membership remains 1.

### Example:

Temperature "High" defined by (30, 35, 45, 50)

For input **x = 40**, it lies in the flat top.

$$\mu(40) = 1$$

For **x = 32**, between 30 and 35:

$$\mu(32) = \frac{32-30}{35-30} = \frac{2}{5} = 0.4$$

# 3. Gaussian Membership Function (GaussMF)

### Shape:

Bell-shaped curve. Very smooth.

### Formula:

$$\mu(x) = e^{-\frac{(x-c)^2}{2\sigma^2}}$$

Where:

- $c$ = center
- $\sigma$ = width

### Example:

Gaussian MF with center 30, σ = 5.

Input: x = 35

$$\mu(35) = e^{-\frac{(35-30)^2}{2(5^2)}} = e^{-\frac{25}{50}} = e^{-0.5} \approx 0.606$$

# 4. Sigmoid Membership Function (SigMF)

### Shape:

S-shaped increasing or decreasing curve.

### Formula:

$$\mu(x) = \frac{1}{1 + e^{-a(x-c)}}$$

Used for:

- Gradually increasing/not sharp boundary sets ("warm", "cool")

| Crisp Input | Fuzzy Sets | Membership Values |
| --- | --- | --- |
| Temp = 32°C | Low, Medium, High | 0.1, 0.6, 0.3 |

---

# 2. Rule Base

### What it contains:

A set of **IF–THEN** fuzzy rules written in natural language.

Example rules:

- IF temperature is High THEN fan speed is Fast
- IF temperature is Low THEN fan speed is Slow

---

# 3. Inference Engine

### What it does:

- Reads rules from the Rule Base
- Applies them to the fuzzified inputs
- Combines the results from all rules

Two common methods:

- **Mamdani Inference**
- **Sugeno Inference**

---

# 4. Defuzzification

### What it does:

Converts fuzzy output obtained from rules into a **single crisp value** (numerical result).

Methods:

- Centroid (most common)
- Mean of Maximum

- Weighted average

Defuzzification converts the **fuzzy output** (multiple fuzzy sets with degrees) into a **single crisp numerical value**.

### Why needed?

- Machines need exact actions (e.g., set fan speed = 72%)

# Main Defuzzification Methods

---

# 1. Centroid Method (Center of Gravity)

**Most popular.**

### Formula:

$$z = \frac{\int \mu(z) \cdot z \, dz}{\int \mu(z) \, dz}$$

### Simple Example:

Output fuzzy sets after inference:

- Medium speed = 0.3
- High speed = 0.7

Corresponding crisp values:
Medium = 50
High = 80

$$z = \frac{(0.3 \times 50) + (0.7 \times 80)}{0.3 + 0.7}$$

$$z = \frac{15 + 56}{1} = 71$$

**Final Output:**

**Fan speed = 71%**

# 2. Mean of Maximum (MoM)

Choose the **average of all points** that have **maximum membership** value.

**Example:**

Output membership values:

- Slow = 0.4
- Medium = 0.6
- Fast = 0.6

Maximum = 0.6 → Medium and Fast

Representative values: Medium = 50, Fast = 80

$$z = \frac{50 + 80}{2} = 65$$

# 3. Largest of Maximum (LoM)

Take the **largest value** with the maximum membership.

From above example:
Max membership = 0.6 → Medium, Fast
Largest → **80**

**Output = 80**

---

# 4. Smallest of Maximum (SoM)

Take the **smallest** of the max membership values.
From same example:

Smallest → **50**

**Output = 50**

---

# Example: Fuzzy Logic for Fan Speed Control

### Problem:

We want to set fan speed based on **room temperature**.

### Input:

Temperature = **30°C**

### Output:

Fan Speed (Slow, Medium, Fast)

---

# Step 1: Fuzzification

Define Membership Functions:

### Temperature MFs:

- Low
- Medium
- High

Assume at 30°C:

- Low = 0.0
- Medium = 0.3
- High = 0.7

So the input belongs **30% to Medium** and **70% to High**.

---

# Step 2: Rule Base

Sample fuzzy rules:

1. IF temperature is Low THEN fan speed is Slow
2. IF temperature is Medium THEN fan speed is Medium
3. IF temperature is High THEN fan speed is Fast

---

# Step 3: Inference Engine

Apply rules based on membership values:

- Rule 2 triggered with strength **0.3** → Medium speed (0.3)
- Rule 3 triggered with strength **0.7** → Fast speed (0.7)

Combine them (aggregation):

- Output fuzzy set =
  Medium (0.3) + Fast (0.7)

---

# Step 4: Defuzzification

Use **Centroid Method**.

Assume numerical values:

- Slow = 20
- Medium = 50
- Fast = 80

Compute final crisp output:

$$\text{Fan Speed} = \frac{(0.3 \times 50) + (0.7 \times 80)}{0.3 + 0.7}$$
$$= \frac{15 + 56}{1} = 71$$

**Final Output:**

**Fan Speed = 71% (Fast)**

# Washing Machine Wash Time

**Inputs:**

1. **Dirt Level** → Low, Medium, High
2. **Load Weight** → Small, Medium, Large

**Output:**

**Wash Time** → Short, Medium, Long

---

# 1. Fuzzification (Convert crisp inputs into fuzzy values)

Assume the washing machine senses:

- **Dirt Level = 7 (scale 0–10)**
- **Load Weight = 5 kg**

Define simple triangular membership functions:

**Dirt Level**

- Low: (0, 0, 4)
- Medium: (3, 5, 7)
- High: (6, 10, 10)

### Load Weight

- Small: (0, 0, 3)
- Medium: (2, 5, 8)
- Large: (6, 10, 10)

### Calculate Memberships

#### *Dirt Level = 7*

- Low(7) = 0
- Medium(7) = 0
- High(7) → On rising slope (6 to 10)

$$\mu_{High} = \frac{7-6}{10-6} = \frac{1}{4} = 0.25$$

#### *Load Weight = 5 kg*

- Small(5) = 0
- Medium(5) = peak → **1**
- Large(5) = 0

**Fuzzified Output:**

- Dirt: High = 0.25
- Weight: Medium = 1

---

# 2. Rule Base (IF–THEN rules)

1. If **Dirt = Low** AND **Weight = Small** → Wash Time = Short
2. If **Dirt = Medium** AND **Weight = Medium** → Wash Time = Medium
3. If **Dirt = High** AND **Weight = Large** → Wash Time = Long
4. If **Dirt = High** AND **Weight = Medium** → Wash Time = Long
5. If **Dirt = Medium** AND **Weight = Large** → Wash Time = Long

(The washing machine contains many such rules.)

---

# 3. Inference Engine (Apply rules using membership values)

Use **min** operator for AND.

Given memberships:

- Dirt High = 0.25
- Weight Medium = 1

Only one rule matches:

### Rule Fired:

**If Dirt = High AND Weight = Medium → Wash Time = Long**

Firing strength:

$$\text{Strength} = \min(0.25, 1) = 0.25$$

Only **Long Wash Time** is activated with strength **0.25**

We use **triangular membership functions** because:

- They are simple
- Easy for exam writing
- Easy to compute
- Most textbooks use triangular MFs for teaching

Using triangular membership formula:

$$\mu(x) = \begin{cases} 0 & x \le a \text{ or } x \ge c \\ \frac{x-a}{b-a} & a < x \le b \\ \frac{c-x}{c-b} & b < x < c \end{cases}$$

Example: Dirt Level = **7**

High Dirt triangle = (6, 10, 10)

Since 6 < 7 < 10:

$$\mu_{High}(7) = \frac{7-6}{10-6} = \frac{1}{4} = 0.25$$

This is why the membership value was **0.25**.

# 4. Defuzzification (Convert fuzzy wash time → crisp minutes)

Assign crisp output values:

- Short = 20 minutes
- Medium = 40 minutes
- Long = 60 minutes

Since only **Long** fires:

$$\text{Wash Time} = \frac{0.25 \times 60}{0.25} = 60 \text{ minutes}$$

When we apply **Weighted Average (Centroid method)**:

$$\text{Final Output} = \frac{\sum(\mu_i \times \text{crisp value}_i)}{\sum \mu_i}$$

The **μ_i** are the membership values.

The **crisp value_i** must be selected by the designer → **(20, 40, 60)**.

# Final Output

**The washing machine sets the wash time to approximately 60 minutes.**

## How the Values Were Chosen

To design a fuzzy logic example, we need four things:

1. **Range of input values (Universe of Discourse)**
2. **Linguistic terms (Low, Medium, High, etc.)**
3. **Membership function shapes (triangular here)**
4. **Peak and boundary points (the numbers like 0, 4, 7, 10)**

# Fuzzy set operations

## Universe of values

U={20, 25, 30, 35, 40}

## Fuzzy Set A = “Warm”

A={(20,0.1),(25,0.4),(30,0.7),(35,1.0),(40,0.8)}

## Fuzzy Set B = “Hot”

B={(20,0.0),(25,0.2),(30,0.5),(35,0.8),(40,1.0)}

1. UNION (A ∪ B)

$$\mu_{A\cup B}(x) = \max(\mu_A(x), \mu_B(x))$$

| x | A(x) | B(x) | max | A ∪ B |
|---|---|---|---|---|
| 20 | 0.1 | 0.0 | **0.1** | (20, 0.1) |
| 25 | 0.4 | 0.2 | **0.4** | (25, 0.4) |
| 30 | 0.7 | 0.5 | **0.7** | (30, 0.7) |
| 35 | 1.0 | 0.8 | **1.0** | (35, 1.0) |

| x | A(x) | B(x) | max | A ∪ B |
|---|---|---|---|---|
| 40 | 0.8 | 1.0 | **1.0** | (40, 1.0) |

## Union (A ∪ B)

{(20,0.1),(25,0.4),(30,0.7),(35,1.0),(40,1.0)}

2. INTERSECTION (A ∩ B)

$$\mu_{A\cap B}(x) = \min(\mu_A(x), \mu_B(x))$$

| x | A(x) | B(x) | min | A ∩ B |
|---|---|---|---|---|
| 20 | 0.1 | 0.0 | **0.0** | (20, 0.0) |
| 25 | 0.4 | 0.2 | **0.2** | (25, 0.2) |
| 30 | 0.7 | 0.5 | **0.5** | (30, 0.5) |
| 35 | 1.0 | 0.8 | **0.8** | (35, 0.8) |
| 40 | 0.8 | 1.0 | **0.8** | (40, 0.8) |

## Intersection (A ∩ B)

{(20,0.0),(25,0.2),(30,0.5),(35,0.8),(40,0.8)}

3. COMPLEMENT (A′)

$$\mu_{A'}(x) = 1 - \mu_A(x)$$

| x | A(x) | 1 – A(x) | A′ |
|---|---|---|---|
| 20 | 0.1 | **0.9** | (20, 0.9) |
| 25 | 0.4 | **0.6** | (25, 0.6) |
| 30 | 0.7 | **0.3** | (30, 0.3) |
| 35 | 1.0 | **0.0** | (35, 0.0) |
| 40 | 0.8 | **0.2** | (40, 0.2) |

## Complement (A′)

{(20,0.9),(25,0.6),(30,0.3),(35,0.0),(40,0.2)}

4. DIFFERENCE (A – B)

$$\mu_{A-B}(x) = \min(\mu_A(x),\ 1 - \mu_B(x))$$

| x | A(x) | B(x) | 1 − B(x) | min(A(x), 1−B(x)) | A − B |
|---|---|---|---|---|---|
| 20 | 0.1 | 0.0 | 1.0 | **0.1** | (20, 0.1) |
| 25 | 0.4 | 0.2 | 0.8 | **0.4** | (25, 0.4) |
| 30 | 0.7 | 0.5 | 0.5 | **0.5** | (30, 0.5) |
| 35 | 1.0 | 0.8 | 0.2 | **0.2** | (35, 0.2) |
| 40 | 0.8 | 1.0 | 0.0 | **0.0** | (40, 0.0) |

A−B={(20,0.1),(25,0.4),(30,0.5),(35,0.2),(40,0.0)}

# 5. CARTESIAN PRODUCT (A × B)

The Cartesian product combines every element of A with every element of B.

Formula:

$$\mu_{A\times B}(x,y) = \min(\mu_A(x),\ \mu_B(y))$$

Use only first two elements of each set:

A = { (20, 0.1), (25, 0.4) }
B = { (20, 0.0), (25, 0.2) }

Now compute **A × B**:

### All pairs (x,y):

1. (20, 20): min(0.1, 0.0) = **0.0**
2. (20, 25): min(0.1, 0.2) = **0.1**
3. (25, 20): min(0.4, 0.0) = **0.0**
4. (25, 25): min(0.4, 0.2) = **0.2**

### Final Cartesian product:

A×B={((20,20),0.0), ((20,25),0.1), ((25,20),0.0), ((25,25),0.2)}